Data Visualization and Exploration

COMS7056A/COMS4060A

Muhammad Nasir

Lecture 3

Lesson Plan

- Transformations
- Feature Engineering
- Key references:
  - Data Mining 3rd Edition, Chapter 7, by I. Witten, E. Frank, & M. Hall
  - Feature Engineering for Machine Learning and Data Analytics, Chapter 8, by G. Dong & H. Liu

Motivation

- Better data beats fancier algorithms.
- Just because the information is technically already in your dataset does not mean a machine learning algorithm will be able to pick up on it.
- **Feature engineering:** We create new features from raw data to increase the predictive power of our final model. These new features should capture extra information that is not obvious in the original features.
- **Feature selection:** The process of selecting the most informative subset of features and reduce the dimensionality of the problem.

Motivation

- Isn’t more data always better?
- Not necessarily
- Sometimes there is pure noise in your data, and this can actually harm your model
- Can’t the algorithm select features and ”learn” transformations?
- Sometimes it can but it helps if you can give it some domain knowledge.
- Sometimes it can’t. Maybe you have some features which are noise.
- It’s also computationally expensive to learn features
- We perform the engineering first to generate additional features, and then feature selection to eliminate irrelevant, redundant, or highly correlated features
- Dimensionality reduction is closely related to feature selection

Preprocessing and transformations

- Before we get into feature engineering, we can apply some transformations to the data to get it into a new representation, in particular, a different shape.
- Why would we make it normal or linear?
  - easier to understand for humans.
  - easier to understand for ML models.
- For example, if you want to calculate confidence intervals, it's much easier to do on a normally distributed dataset – if your data is skewed, you can transform it first to make it normal.
- Transformations should be invertible – you can go back to the original distribution.

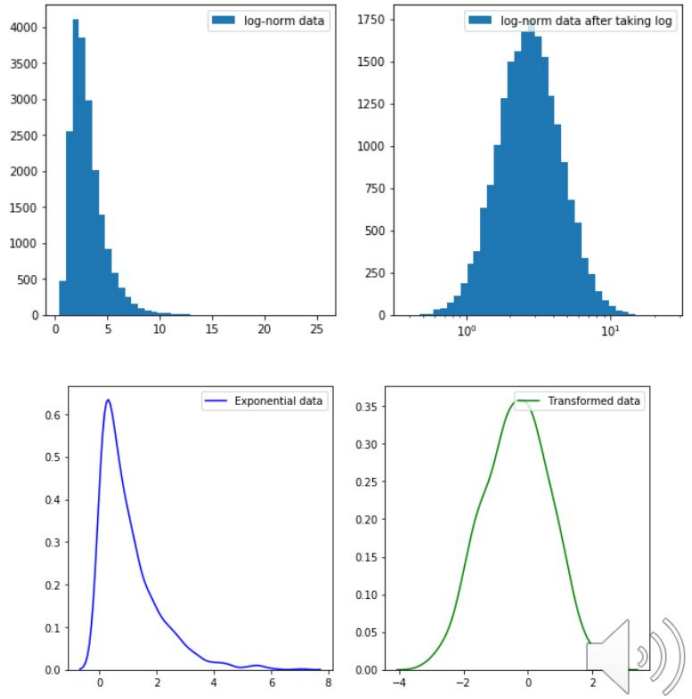
Preprocessing and transformations

- Some algorithms require a normal or standardized input
  - E.g. tree-based classifiers are insensitive to this
  - Nearest neighbours and distance-based algorithms are sensitive to scaling
  - Neural nets have some robustness, but generally, you will want to centre and scale the variables
- Univariate: Box-cox, cube or square a variable, log it, square root, logit
- Multivariate: PCA and ICA (preprocessing or main step)

Preprocessing and transformations

- Log transform:
  - Use if the data spans several orders of magnitude
  - Data can be log-normally distributed
- Box-cox transformation:
  - Only used if you have positive values.

$$y(\lambda) = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0; \\ \log y, & \text{if } \lambda = 0. \end{cases}$$



Box, G. E. P. and Cox, D. R. (1964). An analysis of transformations, Journal of the Royal Statistical Society, Series B, 26, 211-252.

Preprocessing and transformations

- Grouping and aggregations
  - If there is not much data, it can be useful to aggregate the data or group it together – if it's numerical
  - Example: We have bus arrival times, and we want to know at what time of day they are late. The bus arrival times are given down to the second. This is too granular, we should rather look at how many buses arrive in 5 minute periods, or 10 minute periods
  - We would also do this on categorical data using a pivot table.
  - Also a feature engineering step.

Standardisation and Normalisation

- **Standardisation:** Many ML algorithms require the data to be more or less normal; they might give spurious results if individual features do not more or less look like standard normally distributed data, ie, Gaussian with zero mean and unit variance.

$$x' = \frac{x - \mu}{\sigma}$$

- **Normalisation:** Individual features are scaled to have unit norm. This is useful for distance metrics, so that everything is on a similar scale. Also means that we can use algorithms that are sensitive to scale (nearest neighbours, some neural nets work better on values constrained between 0 and 1)

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Standardisation and Normalisation

- **Feature binarization:** Use a threshold on a numerical value to assign a Boolean value. This can be useful for probabilistic estimators that make assumption that the input data is distributed according to a multivariate Bernoulli distribution.
    - For example: binarization of pixels for classification.

Feature engineering

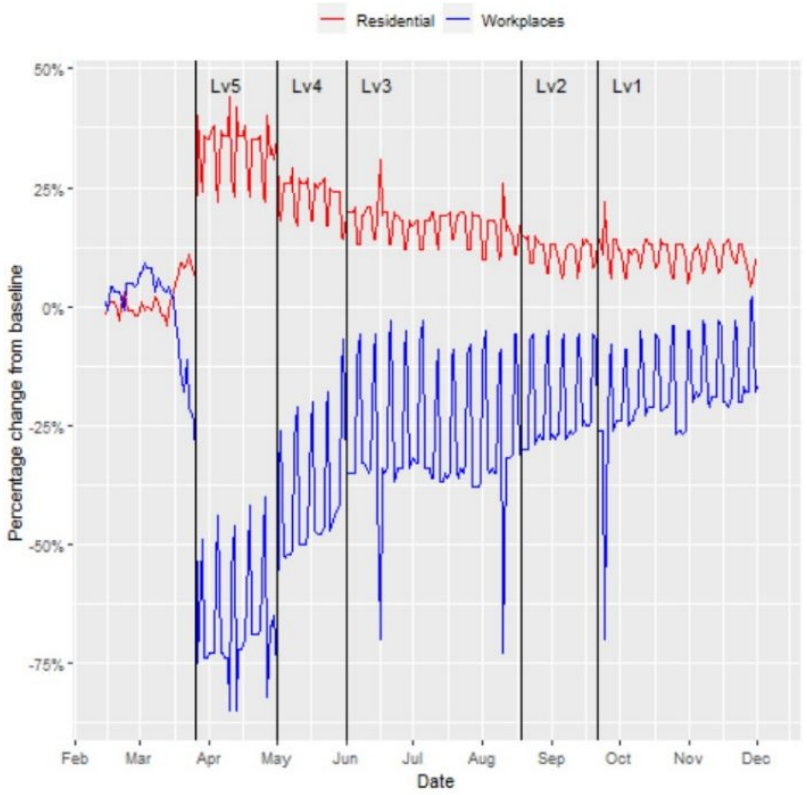
- Too little data or sparse data means that a model will easily overfit – to reduce the variance, we take the existing features and try to combine them into similar features.
- Existing features might contain additional information or more granular data.
- Introducing domain knowledge – if we know something about the dataset that isn't implicit, we might want to add it in
- Sometimes new attributes need to be added to present data in a form that can be used for a specific type of algorithm (we will look at PCA and partial least squares regression as examples in another lecture)
- Feature engineering is domain specific to a large degree – time series or geospatial data have their own unique methods, for example
    - Time series: Seasonality, Fourier coefficients
    - Geospatial: geocoding, determining distances

Indicator Variables

- Indicator values can be created to highlight key metrics or values that incorporate domain knowledge. These new indicators can either be included as a feature, or be used to select or group the remaining data in different ways.
- **Using thresholds:**
  - Looking at Covid case numbers amongst a sample of people, a threshold could be applied to the age to indicate which people are eligible for vaccination at a specific point in time.
- **Combination of multiple variables:**
  - Investigating housing prices, and every house has multiple features, such as number of bedrooms and bathrooms. We know that houses or apartments that have exactly 2 bedrooms and 1 bathroom attract high rental prices (which would potentially make it a more attractive buy), which we could now highlight as an additional variable in our dataset.
- **Special events:**
  - You're modeling foot traffic (or real traffic) for a mall (or the traffic department). You want to know how traffic changes by day of week, time of day, or day of month. We expect that traffic will change according to some external factors too – lockdown, public holidays, riots. We could create indicators for these specific data points in any analysis that tell us they are not "normal" values, and should be handled differently.
- **Groups of classes/categorical values:**
  - You're analysing new clients coming to your company, and there is a variable that says how they signed up, or the source of their referral. You could create an indicator variable to flag those that come from a online vs telephonic.

Example Indicator

- During Covid lockdown levels, mobility trends changed drastically.
- Here the dates of the lockdown have been added to the additional dataset and plotted
- What might have been considered an anomaly was explained by domain / user knowledge



https://towardsdatascience.com/how-has-mobility-changed-in-south-africa-d91e234b4b8c

Decomposition Of Variables

- An existing feature can contain additional information that could be useful in modelling.
  - If we are looking at the bus arrival times, imagine that the data is given in the format 2025-07-31 13:02:08.531731. We are not interested in the exact second, but perhaps the minute, hour, date, day, or some 15 minute interval.
  - We could convert the date into categorical attributes like hour_of_the_day, part_of_day, timezone.
- In the Titanic dataset, extracting the title from the name can give value information: Mr, Mrs, Miss, Count, etc. See the next slide.

Titanic decomposition

[10]:
```
df.head()
```

[10]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

We can get the titles of the passengers - this might have some useful information about ages and socio-economic class.

[26]:
```
df['name_list'] = df.Name.apply(lambda x: x.split(','))
# get the title from the Name
df['title'] = df.name_list.apply(lambda x: x[1].strip().split(' ')[0])
# get the surname
df['surname'] = df.name_list.apply(lambda x: x[0])
# first name
df['first_name'] = df.name_list.apply(lambda x: x[1].strip().split(' ')[1])

# do it for the test ds too
df_test['name_list'] = df_test.Name.apply(lambda x: x.split(','))
# get the title from the Name
df_test['title'] = df_test.name_list.apply(lambda x: x[1].strip().split(' ')[0])
# get the surname
df_test['surname'] = df_test.name_list.apply(lambda x: x[0])
# first name
df_test['first_name'] = df_test.name_list.apply(lambda x: x[1].strip().split(' ')[1])


print(df.title.value_counts())
```

```
Mr.          517
Miss.        182
Mrs.         125
Master.       40
Dr.            7
Rev.           6
Col.           2
Mlle.          2
Major.         2
the            1
Don.           1
Capt.          1
Ms.            1
Mme.           1
Sir.           1
Jonkheer.      1
Lady.          1
Name: title, dtype: int64
```

Note that some of the titles have very few values - these are probably higher classes, but let's group them together as a first pass.

[35]:
```
df['title'] = np.where(df['title'].isin(['Mr.', 'Mrs.', 'Master.', 'Miss.']), df.title, 'Other')
print(df.title.value_counts())
```

```
Mr.        517
Miss.      181
Mrs.       124
Master.     40
Other       27
Name: title, dtype: int64
```

Dealing with sparse classes

- **Sparse classes** (in categorical features) are those that have very few total observations. They can be problematic for certain machine learning algorithms, causing models to be overfit. In this case, we have to combine classes together (as we did with titles in the example earlier, or video game publishers)

Categorical data

- Categorical features refer to qualitative values, or values like names, strings.
- Often necessary to convert all categorical features (strings, essentially) to numeric values or binary values so that a ML model can handle them.
- Usually, we will use either of the below:
  - **One Hot Encoding** takes a categorical feature and splits it in as many columns as there are categories. All the rows which do not have that category are labelled 0, and 1 for all those with the category. See get_dummies() function in Pandas.
  - **Label Encoding** replaces all the categorical values with a different number and stores it in a single column.
    - NOTE: Some ML models might incorrectly infer that the encoded cases with higher values might be more important (ie that they are in hierarchical order). This can be avoided by using one-hot encoding.
    - If hierarchy is present then it is better to use label encoding: e.g, “good” = 2, “okay” = 1, “bad” = 0.

One-Hot Encoding

[8]:
```
df = pd.read_csv('vgsales.csv')
```

[9]:
```
df.head()
```

[9]:

	Rank	Name	Platform	Year	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales
0	1	Wii Sports	Wii	2006.0	Sports	Nintendo	41.49	29.02	3.77	8.46	82.74
1	2	Super Mario Bros.	NES	1985.0	Platform	Nintendo	29.08	3.58	6.81	0.77	40.24
2	3	Mario Kart Wii	Wii	2008.0	Racing	Nintendo	15.85	12.88	3.79	3.31	35.82
3	4	Wii Sports Resort	Wii	2009.0	Sports	Nintendo	15.75	11.01	3.28	2.96	33.00
4	5	Pokemon Red/Pokemon Blue	GB	1996.0	Role-Playing	Nintendo	11.27	8.89	10.22	1.00	31.37

[10]:
```
pd.get_dummies(df.Genre)
```

[10]:

	Action	Adventure	Fighting	Misc	Platform	Puzzle	Racing	Role-Playing	Shooter	Simulation	Sports	Strategy
0	0	0	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	1	0	0	0	0	0	0	0
2	0	0	0	0	0	0	1	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	1	0
4	0	0	0	0	0	0	0	1	0	0	0	0
...	...	...	...	...	...	...	...	...	...	...	...	...
16593	0	0	0	0	1	0	0	0	0	0	0	0
16594	0	0	0	0	0	0	0	0	1	0	0	0
16595	0	0	0	0	0	0	1	0	0	0	0	0
16596	0	0	0	0	0	1	0	0	0	0	0	0
16597	0	0	0	0	1	0	0	0	0	0	0	0

16598 rows × 12 columns

https://www.kaggle.com/gregorut/videogamesales

High Cardinality Categorical Data

- If the dataset has many unique categories, then dummy (one-hot) encoding might increase the number of attributes too much (eg. Zip codes).
- In this situation, we need another approach – transform it into a single continuous variable correlated with the target label.
- Can apply a continuous transformation, such that each category gets a score
  - Supervised Ratio: probability that the ith value of variable X belongs to the positive class:
    - $SR_i = \frac{P_i}{N_i + P_i}$ Where P_i is the number of positive labels, N_i is number of negative labels in total
  - Weight of evidence: For each class i of an independent variable x, we want to find the ratio of the proportion of the population, whose dependent variable y belongs to a certain class, that has the class i.
  - There are many so-called “Deep Features” described on the featuretools website too, which achieve a similar outcome
- Apply a binning technique: eg. IP ranges could be used instead of individual IP addresses
- For zip codes, geocode it to get a suburb, instead of many individual zip codes
- Some categories are too sparse to use

https://featuretools.alteryx.com/en/stable/

Weight of Evidence example

- Positive weight of evidence means higher percentage of events. Of course, the definition of event/non event is a decision you need to make. What this means here, is that there is a higher-than-average death rate in males on the Titanic.

```
# calculating the weight of evidence for survival for the category Sex
male_event = len(df[(df.Survived == 1) & (df.Sex == 'male')]) / len(df[df.Survived == 1])
male_non_event = len(df[(df.Survived == 0) & (df.Sex == 'male')]) / len(df[(df.Survived == 0)])
woe_male = np.log(male_non_event/male_event)
print(woe_male)

female_event = len(df[(df.Survived == 1) & (df.Sex == 'female')]) / len(df[df.Survived == 1])
female_non_event = len(df[(df.Survived == 0) & (df.Sex == 'female')]) / len(df[(df.Survived == 0)])
woe_female = np.log(female_non_event/female_event)
print(woe_female)

df['sex_woe'] = np.where(df.Sex == 'male', woe_male, woe_female)
```

```
0.9779675897891794
-1.5271213797486785
```

Discretisation methods

- **Representation**: The idea here is to split a variable into a binary or nominal value (eg. < 5 and >= 5). (A decision tree does this implicitly, multiple times, creating a new split at each node in the tree). We are going to create nominal, ordered values, like one-hot encoding.
- **Unsupervised**: equal-width binning distributes unevenly between classes, rather use equal-frequency binning. If there are classes too, then this can be taken into account when creating the bins, otherwise, this can cause imbalances. Check out KBinsDiscretizer in SKLearn if you are using python.
- **Entropy**: Similar idea to the way it works for the formation of a decision tree – recursively splitting intervals until some stopping criteria is met. At each point in the algorithm, it looks for the split that will offer the greatest information gain. (See pg 316/7 in Data Mining book)

Binning of Ages: Titanic

```
# let's do a basic one where we use 5 bins of ages:
df['age_bin'] = pd.cut(df.Age.astype(int), 5)
df_test['age_bin'] = pd.cut(df.Age.astype(int), 5)


print(df.age_bin.value_counts())

## let's do a basic one where we use 5 bins of ages:
df['fare_bin'] = pd.qcut(df.Fare, 5)
df_test['fare_bin'] = pd.qcut(df.Fare, 5)

# print (pd.cut(df.Age.astype(int), 5))
```

```
(16.0, 32.0]     495
(32.0, 48.0]     216
(-0.08, 16.0]    100
(48.0, 64.0]      69
(64.0, 80.0]      11
Name: age_bin, dtype: int64
```

```
df2[['PassengerId', 'age_bin_encoded_0', 'age_bin_encoded_1', 'age_bin_encoded_2', 'age_bin_encoded_3', 'age_bin_encoded_4']]
```

	PassengerId	age_bin_encoded_0	age_bin_encoded_1	age_bin_encoded_2	age_bin_encoded_3	age_bin_encoded_4
0	1	0.0	1.0	0.0	0.0	0.0
1	2	0.0	0.0	1.0	0.0	0.0
2	3	0.0	1.0	0.0	0.0	0.0
3	4	0.0	0.0	1.0	0.0	0.0
4	5	0.0	0.0	1.0	0.0	0.0
...	...	...	...	...	...	...
886	887	0.0	1.0	0.0	0.0	0.0
887	888	0.0	1.0	0.0	0.0	0.0
888	889	0.0	1.0	0.0	0.0	0.0
889	890	0.0	1.0	0.0	0.0	0.0
890	891	0.0	1.0	0.0	0.0	0.0

889 rows × 6 columns

Feature crossing / variable interaction

- Creating new features as a combination of existing features enables us to capture interactions between variables. Could be multiplying numerical features, or combining categorical variables. This is a great way to add domain expertise knowledge to the dataset.
- **Sum of two features:** Looking at the number of vehicles sold (or predicted to be sold) in a month. You have the features sales_bakkies and sales_cars. You could sum those features if you only care about overall sales
- **Difference between two features:** Looking at the housing market, you might have the date a house was listed, and the date it was purchased. You can take their difference to create a feature for length
- **Product of two features:** You have the expected uptake of a product and the profit you expect per sale. You can take their product to create the feature earnings.
- **Quotient of two features:** You have the number of clients, how much they spend, and the day of week they spend it on. You can calculate spend per client per day as a new variable.

Data Augmentation

- Time series data:
  - Time series data can be augmented using a single feature, some form of date, to layer in features other datasets
  - Eg. Combining weather data with driving behaviour on a given day.
- Geocoding:
  - Geocode a street address or point of interest into a latitude and longitude. This can then be used to calculate distances between places using the haversine formula. (You can also go the other direction, from lat, long into a street address). This can be done online with Mapbox, Google Maps, and OpenStreetMaps.
- Augment by clustering your data
  - Giving synthetic cluster labels can be effective.

Text Data

- Sometimes your dataset contains sentences. e.g. Banking transactions.
- Word-To-Vec can provide embeddings.
- LLMs like BERT are famous for embeddings.
- Cluster the embeddings.
- Cluster assignment becomes a categorical feature.

Before using a new feature…

- Can this feature be computed for future observations?
- Is it intuitive or easy to explain?
- Is it based on domain knowledge or exploratory analysis?
- Is it predictive?
- **Never touch the target variable.** Do not build features from the target variable. In production, this will not be available. Considered data leakage!

Next Lecture

- Feature Selection